ZYRO — Complete WhatsApp Embedded Signup v4 Integration

Project Objective

Implement a complete, production-ready, multi-tenant WhatsApp Business Platform integration in my existing ZYRO SaaS application using Meta's WhatsApp Embedded Signup v4.

The purpose is to allow every ZYRO merchant to connect their own WhatsApp Business account to ZYRO through Meta's official onboarding flow.

Merchants should not need to create their own Meta Developer applications, configure webhooks, or manually provide technical credentials such as:

- Meta App ID
- Meta App Secret
- WhatsApp Business Account ID (WABA ID)
- Phone Number ID
- System User Access Token
- Webhook URL
- Webhook Verify Token

ZYRO owns and manages the Meta application and integration infrastructure centrally. Each merchant independently authorizes access to their own WhatsApp Business account.

IMPORTANT: Work inside my existing GitHub repository. Do not create a new project or replace the existing application.

---

1. First Analyze the Existing Project

Before writing or modifying any code:

1. Inspect the entire existing project structure.
2. Identify the Next.js version and whether it uses the App Router or Pages Router.
3. Identify the existing Supabase client and server-side authentication helpers.
4. Inspect how user authentication works.
5. Inspect the existing `users_table` structure and determine how the authenticated Supabase user maps to a merchant.
6. Inspect existing API routes, middleware, environment variable conventions, database utilities, and UI components.
7. Locate the existing WhatsApp webhook implementation.
8. Preserve all existing webhook verification, message reception, and signature validation functionality.
9. Check whether WhatsApp-related database tables already exist before creating new ones.
10. Identify existing dashboard navigation and settings pages where WhatsApp integration should appear.

Do not assume that the project follows a standard Next.js folder structure.

Do not create duplicate Supabase clients, authentication systems, database tables, or API routes.

Do not overwrite existing functionality.

If an essential architectural detail is unclear, explain what you found and ask me before making a potentially destructive change.

After analyzing the project, provide a short implementation plan and then proceed with the implementation.

---

2. Existing Technology Architecture

Use the existing technology stack:

- Frontend: Next.js
- Backend: Next.js API routes / Route Handlers
- Language: JavaScript
- Database: Supabase PostgreSQL
- Authentication: Supabase Auth
- Hosting: Vercel
- Meta integration: WhatsApp Business Platform Cloud API
- Meta onboarding: Embedded Signup v4

Important requirements:

- Use JavaScript only.
- Do not introduce TypeScript.
- Do not introduce a separate backend server.
- Do not introduce an additional authentication provider.
- Do not move the application away from Vercel.
- Use the existing Supabase database.
- Use the existing Supabase authentication session.
- Use server-side API routes for sensitive operations.
- Keep all secrets server-side.
- Do not expose access tokens or Meta App Secret to the browser.

The integration must support multiple independent merchants.

---

3. Meta Application Configuration

ZYRO already has its own Meta Developer application.

Use the following existing configuration:

META_APP_ID:

2070163980283029

META_CONFIG_ID:

2329644037853553

META_REDIRECT_URI:

https://www.zyroocloud.com/meta-callback

META_HOSTED_SIGNUP_URL:

https://business.facebook.com/messaging/whatsapp/onboard/?app_id=2070163980283029&config_id=2329644037853553&extras=%7B%22version%22%3A%22v4%22%2C%22sessionInfoVersion%22%3A%223%22%2C%22featureType%22%3A%22whatsapp_business_app_onboarding%22%7D&redirect_uri=https%3A%2F%2Fwww.zyroocloud.com%2Fmeta-callback

Signup configuration:

- Embedded Signup version: v4
- Session Info Version: 3
- Feature Type: whatsapp_business_app_onboarding
- Redirect URI: https://www.zyroocloud.com/meta-callback

Graph API version should be configurable through an environment variable:

META_GRAPH_API_VERSION

Use a currently supported version compatible with the Meta application. Do not silently assume an unsupported version.

All Meta credentials must be read from environment variables.

Required environment variables:

```
META_APP_ID= META_APP_SECRET= META_CONFIG_ID= META_GRAPH_API_VERSION=
META_REDIRECT_URI=  META_WEBHOOK_VERIFY_TOKEN=  SUPABASE_URL=
SUPABASE_ANON_KEY= SUPABASE_SERVICE_ROLE_KEY=
WHATSAPP_TOKEN_ENCRYPTION_KEY=
```

Use the existing environment variable naming conventions if equivalent variables already exist.

Never create a NEXT_PUBLIC variable for:

- META_APP_SECRET
- SUPABASE_SERVICE_ROLE_KEY
- WhatsApp access tokens
- WHATSAPP_TOKEN_ENCRYPTION_KEY

Do not ask me to paste secrets into source code or commit them to GitHub.

---

4. Multi-Merchant Architecture

This is a critical requirement.

ZYRO must support multiple merchants connecting different WhatsApp Business accounts.

Example:

Merchant A:

- User ID: A
- WhatsApp Business Account: WABA-A
- Phone Number ID: PHONE-A
- Access Token: TOKEN-A

Merchant B:

- User ID: B
- WhatsApp Business Account: WABA-B
- Phone Number ID: PHONE-B
- Access Token: TOKEN-B

Merchant C:

- User ID: C
- WhatsApp Business Account: WABA-C
- Phone Number ID: PHONE-C
- Access Token: TOKEN-C

Each merchant must only access their own WhatsApp account, conversations, contacts, and messages.

ZYRO must never use one merchant's access token or phone number to send messages on behalf of another merchant.

Use the existing `users_table` as the source of merchant identity.

Determine the relationship between `users_table` and `auth.users` before implementing database queries.

Do not assume that the authenticated user's UUID is the primary key of `users_table` without inspecting the schema.

If a new WhatsApp connection table is required, associate every connection with the correct existing merchant/user record.

Support multiple WhatsApp connections per merchant if the existing product architecture permits it.

---

5. Connect WhatsApp User Interface

Implement a professional WhatsApp integration section inside the existing ZYRO dashboard.

The interface should include:

Disconnected state

Display:

- WhatsApp Business integration title
- WhatsApp logo

- Description explaining the integration
- Connect WhatsApp button
- Short explanation of what happens when the merchant connects

Example:

"Connect your WhatsApp Business account to ZYRO and manage customer conversations, messages, and business communication from one dashboard."

When the merchant clicks Connect WhatsApp:

1. Verify the merchant is authenticated.
2. Initiate a secure signup session.
3. Generate a cryptographically secure random state value.
4. Bind the signup session to the authenticated merchant.
5. Store the state securely, preferably server-side, with a short expiry.
6. Launch the configured Meta Embedded Signup v4 experience.
7. Preserve the correct redirect URI and signup configuration.

Do not trust merchant IDs supplied by the browser.

Connected state

After successful onboarding, display:

- WhatsApp connection status
- Connected phone number
- WhatsApp Business Account information where appropriate
- Business display name, if available
- Connection date
- Disconnect button
- Reconnect option if the connection expires or becomes invalid

Use the existing dashboard design system and styling.

Make the interface responsive for desktop and mobile.

Do not create a disconnected, unrelated UI design.

---

6. Embedded Signup v4 Implementation

Implement Meta's actual Embedded Signup v4 flow.

Important: Distinguish between the Meta-hosted onboarding URL and the JavaScript SDK Embedded Signup flow.

Inspect the configured signup method and implement the correct supported flow. Do not assume that authorization code, WABA ID, and Phone Number ID all arrive in the same callback query parameters.

Use the supplied hosted signup URL if it is the appropriate flow for this configuration.

If the v4 configuration requires the JavaScript SDK to capture session information, implement the supported SDK launch and event/message listener mechanism.

For v4 session information:

- Capture the supported signup completion event.
- Capture the returned WABA ID.
- Capture the returned phone number ID.
- Capture any other identifiers required by the configured onboarding flow.
- Validate all received data server-side.
- Do not trust browser-supplied identifiers without validating access through Meta's Graph API.

The feature type is:

```
whatsapp_business_app_onboarding
```

This is intended for the WhatsApp Business App onboarding / Coexistence flow where eligible.

Do not assume that every merchant's existing WhatsApp Business App number is eligible for Coexistence.

Handle Meta cancellation, errors, rejected permissions, incomplete onboarding, and expired signup sessions gracefully.

---

7. Secure Callback and Authorization Code Exchange

Implement the callback page:

```
/meta-callback
```

Implement the required server-side API routes using the project's existing route conventions.

Expected flow:

1. Merchant starts onboarding from ZYRO.
2. ZYRO creates a signup state.
3. Merchant completes the Meta flow.
4. Meta returns the authorization result to the configured callback.
5. Callback validates the state against the original signup session.
6. Callback sends the authorization code to a secure backend endpoint.
7. Backend verifies the authenticated Supabase user and signup session.
8. Backend exchanges the authorization code with Meta using the App ID and App Secret.
9. Backend validates the response and handles errors.
10. Backend obtains and validates the relevant business integration credentials and asset access according to Meta's current Tech Provider Embedded Signup requirements.
11. Backend associates the resulting WABA and phone number with the authenticated merchant.
12. Backend encrypts and persists the access token.
13. Backend completes the required WABA subscription/onboarding operations.
14. The frontend displays the successful connection.

Security requirements:

- Use cryptographically secure state values.
- Prevent CSRF and replay attacks.
- Enforce a short signup session expiry.
- Validate the current Supabase user.
- Validate the callback state.
- Prevent one merchant from completing another merchant's signup session.
- Never accept a merchant ID as authoritative from the browser.
- Never expose the Meta App Secret.
- Never return the WhatsApp access token to the frontend.
- Never log authorization codes, App Secrets, or access tokens.
- Use server-side Graph API calls.
- Handle Meta API errors without exposing sensitive information.

Do not consider the integration successful just because the authorization code was exchanged.

A connection is successful only after the merchant's business assets and required permissions have been validated and the connection has been securely saved.

---

8. Access Token Storage and Encryption

Implement secure access token storage.

WhatsApp access tokens must be encrypted at rest before being saved in Supabase.

Use AES-256-GCM or another appropriate authenticated encryption implementation.

Use:

```
WHATSAPP_TOKEN_ENCRYPTION_KEY
```

from server-side environment variables.

Requirements:

- Generate a unique IV/nonce for each encryption operation.
- Store the authentication tag.
- Store the encryption version to support future key rotation.
- Decrypt tokens only inside server-side operations that need them.
- Never return decrypted tokens to the browser.
- Never expose them in logs.
- Never store them in localStorage, sessionStorage, or frontend state.

Use the appropriate token type and lifecycle for Meta's Tech Provider Embedded Signup flow.

Do not assume that a normal short-lived OAuth token is automatically a permanent business integration token.

Handle token expiration, invalidation, and reconnection.

---

9. Supabase Database Design

Inspect the existing schema before implementing migrations.

Create only the tables and columns that are actually missing.

The WhatsApp integration should support at least the following logical data:

whatsapp_connections

Suggested fields:

- id
- user_id or merchant_id, referencing the actual existing merchant identity
- waba_id
- phone_number_id
- display_phone_number
- verified_name
- encrypted_access_token
- token_encryption_version
- token_expires_at, if applicable
- connection_status
- onboarding_type
- created_at
- updated_at

Requirements:

- A phone number must not be connected to multiple unrelated merchants.
- Add appropriate unique constraints.
- Support reconnecting and updating existing connections.
- Store only the information required by the application.
- Do not store access tokens as plaintext.

whatsapp_signup_sessions

Use a secure, short-lived server-side record for signup sessions if needed.

Suggested fields:

- id
- user_id
- state_hash
- status
- expires_at
- created_at
- completed_at

Do not store raw state values unnecessarily.

Future message tables

Prepare the architecture to support:

- whatsapp_contacts
- whatsapp_conversations
- whatsapp_messages
- whatsapp_webhook_events

These must all be tenant-scoped.

Do not implement a large new messaging system if the project already has equivalent tables.

Use Supabase Row Level Security.

Ensure one merchant cannot query or modify another merchant's records.

Use the Supabase service role only in trusted server-side code, never in browser code.

If you introduce database migrations, provide the SQL and explain how to apply them.

Do not execute destructive schema changes.

---

10. WABA Subscription and Post-Onboarding Operations

After successful signup, perform the required Meta onboarding operations for this Tech Provider configuration.

Inspect Meta's current official documentation and determine which operations apply.

These may include:

- Retrieving the shared WABA.
- Validating the merchant's WABA access.
- Validating phone number ownership/access.
- Adding or assigning the required business integration system user where required.
- Registering a phone number if required by the selected onboarding flow.
- Subscribing ZYRO's Meta application to the merchant's WABA.
- Retrieving relevant business and phone number metadata.
- Completing any applicable Tech Provider onboarding or billing prerequisites.

Important:

Do not blindly execute standard Cloud API phone-number registration for a Coexistence onboarding flow.

Follow the exact operations required by the selected Embedded Signup feature and the returned session information.

Implement each operation with error handling and safe retry behavior.

Do not mark a merchant as connected if required operations fail.

---

11. Existing Webhook Integration

ZYRO already has a deployed webhook endpoint:

`/api/whatsapp/webhook/messages`

Its public URL is configured in Meta.

The webhook GET verification is already successful.

A test WhatsApp message has already reached the webhook, and the Vercel logs show:

`Incoming WhatsApp message:`

Preserve the current working implementation.

Do not replace or break it.

Enhance it for multiple merchants.

Incoming webhook payloads must be associated with the correct merchant using:

`value.metadata.phone_number_id`

Expected processing:

1. Receive the webhook.
2. Verify the Meta webhook signature using the raw request body and Meta App Secret.
3. Extract the phone number ID.
4. Find the corresponding `whatsapp_connections` record.
5. Identify the merchant.
6. Store the event in a tenant-scoped webhook event table.
7. Process the incoming message or status update.
8. Update the appropriate contact, conversation, and message records.
9. Prevent duplicate event/message processing.
10. Handle unknown phone number IDs safely.

One ZYRO webhook endpoint must support every connected merchant.

Do not create a separate webhook URL for every merchant.

Use idempotency and appropriate database constraints.

---

12. Sending WhatsApp Messages

After the connection is complete, prepare the server-side architecture for sending messages.

A send operation must:

1. Authenticate the current ZYRO user.
2. Validate that the user belongs to the relevant merchant.
3. Retrieve that merchant's WhatsApp connection.
4. Decrypt that merchant's access token server-side.
5. Use that connection's Phone Number ID.
6. Send the request through the correct Graph API endpoint.
7. Store the outgoing message and its WhatsApp message ID.
8. Handle Meta errors.
9. Never allow a merchant to send messages using another merchant's credentials.

Use the appropriate Cloud API message endpoint:

```
POST /{PHONE_NUMBER_ID}/messages
```

Implement this only after the connection model is securely established.

Respect WhatsApp's messaging policies, templates, customer service window, and applicable message restrictions.

---

13. Security and Multi-Tenant Isolation

Security is a priority.

Implement:

- Supabase authentication checks on all protected endpoints.
- Merchant ownership validation.
- Row Level Security policies.
- Server-side access token encryption.
- CSRF protection through signup state validation.
- Webhook signature validation.
- Input validation.
- Rate limiting where appropriate.
- Idempotent webhook processing.
- Safe error handling.
- No secret leakage in browser responses or logs.

Never rely only on frontend checks for merchant access.

Never trust a merchant ID supplied by the frontend without verifying membership.

Never use a global WhatsApp access token for every merchant.

---

14. Error Handling and User Experience

Handle the following scenarios:

- Merchant is not logged in.
- Meta popup or signup is cancelled.
- Merchant denies permissions.
- Signup session expires.
- Invalid authorization code.
- Invalid signup state.
- Meta token exchange fails.
- WABA information cannot be retrieved.
- Phone number validation fails.
- Coexistence is unavailable for the merchant.
- WABA subscription fails.
- Access token becomes invalid.
- Merchant reconnects an existing account.
- The same phone number is already connected to another merchant.
- Network timeout or Meta API outage.

Display clear, user-friendly messages in the dashboard.

Store technical details in secure server logs without leaking credentials.

Allow merchants to retry onboarding where appropriate.

---

15. Testing Requirements

Before considering the implementation complete, test:

1. An authenticated merchant can start signup.
2. An unauthenticated user cannot start signup.
3. Meta signup opens with the correct app and configuration IDs.
4. Callback state validation works.
5. Invalid states are rejected.
6. Authorization code exchange happens server-side.
7. Meta App Secret never reaches the browser.
8. The access token is encrypted before database storage.
9. WABA and phone number are associated with the correct merchant.
10. WABA subscription succeeds where required.
11. A merchant can see their own connection.
12. A merchant cannot see another merchant's connection.
13. The existing webhook continues working.
14. Webhook events are routed to the correct merchant.
15. Duplicate webhook events do not create duplicate messages.
16. Meta API failures are handled gracefully.
17. The application builds successfully for Vercel.

Do not fabricate successful API responses.

If a test requires actual Meta credentials, explain the manual test procedure and identify which part remains unverified.

---

16. Implementation Rules for Google AI Studio

Follow these rules throughout the task:

1. Work directly in the existing repository.
2. Use JavaScript only.
3. Inspect files before modifying them.
4. Reuse existing project conventions.
5. Do not unnecessarily refactor unrelated code.
6. Do not overwrite existing working webhook functionality.
7. Do not create duplicate database tables.
8. Do not invent database column names.
9. Do not hardcode credentials.
10. Do not expose secrets in client components.
11. Do not use mock implementations as production functionality.
12. Do not silently skip required Meta API operations.
13. Do not claim that a merchant is connected before validating the actual connection.
14. Do not implement a separate WhatsApp server.
15. Keep all operations compatible with Vercel serverless execution.
16. Use official Meta documentation for API behavior and permissions.
17. If an API requirement is unclear, investigate it instead of guessing.
18. After each major implementation stage, explain which files were created or modified.
19. Provide any required Supabase SQL migrations separately.
20. Run available lint/build checks and fix errors caused by your changes.

Do not stop after creating the Connect WhatsApp button. Implement the complete integration in logical stages.

---

17. Final Expected Result

When the implementation is complete, ZYRO should work as follows:

Merchant A

1. Logs into ZYRO.
2. Opens Settings → WhatsApp.
3. Clicks Connect WhatsApp.
4. Completes Meta Embedded Signup.
5. Authorizes their own WhatsApp Business account.
6. Returns to ZYRO.
7. Sees their connected WhatsApp business number.
8. Can eventually manage WhatsApp conversations through ZYRO.

Merchant B

Follows exactly the same process but connects a completely different WhatsApp Business account and phone number.

Both merchants use:

- The same ZYRO application.
- The same Meta Developer app.
- The same Meta webhook endpoint.
- The same ZYRO backend infrastructure.

But they have separate:

- WhatsApp Business accounts.
- Phone Number IDs.
- Access tokens.
- Contacts.
- Conversations.
- Messages.
- Permissions and connection records.

The final objective is to make WhatsApp integration a self-service feature of ZYRO. A merchant should be able to connect their own WhatsApp Business account without any technical assistance from the ZYRO team.

Begin by analyzing the repository, identifying the existing authentication and database structure, and implementing the integration without disrupting existing functionality.